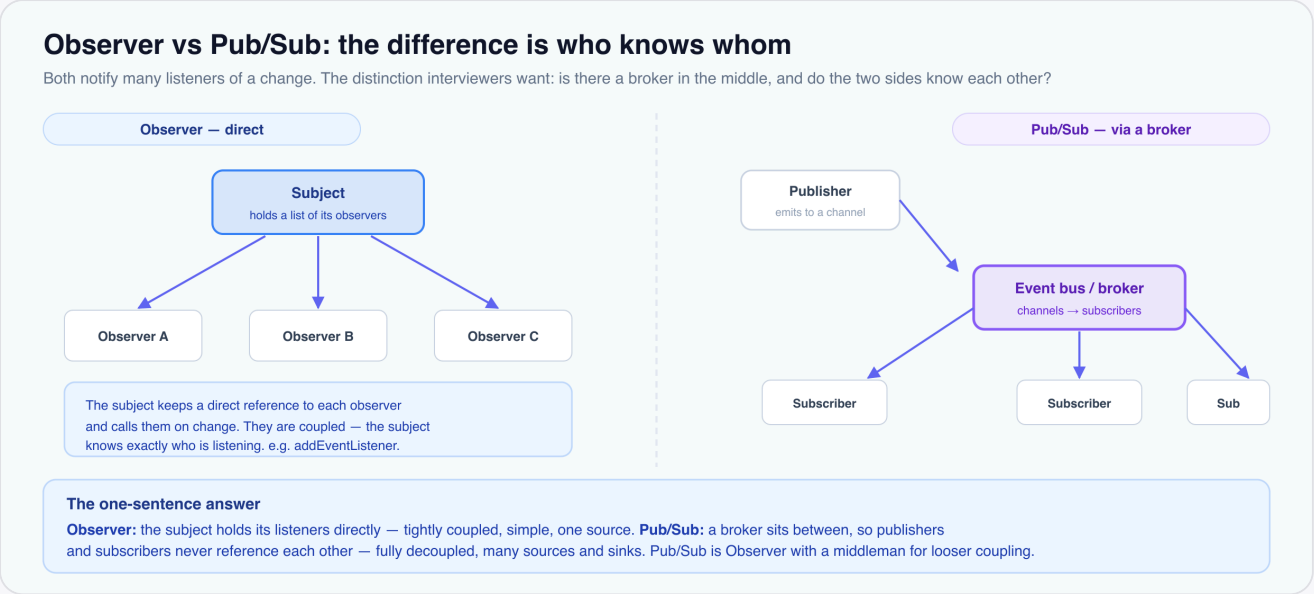
Behavioral patterns

12 min read Updated 16 Jul 2026 Topic 25 of 77 Bookmark Mark done

9B. Behavioral Patterns

Questions: 9 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Microsoft, Atlassian, Swiggy

These are the most interview-relevant patterns. Observer, Pub/Sub, Strategy, and Command map directly to problems you'll be asked to solve.



Q 9.7 What is the Observer pattern? How does it relate to DOM events?

Difficulty: Mid **Asked at:** Every company

Observer defines a one-to-many dependency — when one object (the subject) changes state, all its dependents (observers) are notified automatically.

Frontend System Desi...

JAVASCRIPT

Copy

```javascript
class EventEmitter {
  constructor() {
    this.listeners = new Map();
  }

  on(event, callback) {
    if (!this.listeners.has(event)) {
      this.listeners.set(event, new Set());
    }
    this.listeners.get(event).add(callback);
    return () => this.off(event, callback);  // Return unsubscribe functio
  }

  off(event, callback) {
    this.listeners.get(event)?.delete(callback);
  }

  emit(event, data) {
    this.listeners.get(event)?.forEach(cb => cb(data));
  }

  once(event, callback) {
    const wrapper = (data) => {
      callback(data);
      this.off(event, wrapper);
    };
    this.on(event, wrapper);
  }
}

// Usage
const store = new EventEmitter();

const unsubscribe = store.on('cart:updated', (cart) => {
  renderCartBadge(cart.items.length);
});
```

DOM events ARE the Observer pattern. `addEventListener` registers an observer, the browser is the subject, and when a user clicks, all registered observers are notified.

JAVASCRIPT Copy

```
// This is Observer pattern — button is the subject, handler is the observ
button.addEventListener('click', handleClick);
```

React's `useState` + re-rendering is also Observer — when state changes, all components that use that state are notified (re-rendered).

Follow-up: "What's the difference between Observer and Pub/Sub?"

In Observer, the subject knows about its observers directly. In Pub/Sub, there's a message broker in between — publishers and subscribers don't know about each other. See Q9.8.

Q 9.8 What is the Pub/Sub pattern? How is it different from Observer?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Razorpay

Pub/Sub adds a message broker between publishers and subscribers. Publishers emit events to the broker, subscribers listen on the broker. Neither knows about the other.

JAVASCRIPT Copy

```
// Simple Pub/Sub message bus
const EventBus = {
  channels: new Map(),

  subscribe(channel, callback) {
    if (!this.channels.has(channel)) {
```

```
    return () => this.channels.get(channel)?.delete(callback);
  },

  publish(channel, data) {
    this.channels.get(channel)?.forEach(cb => cb(data));
  },
};

// Header component — publishes
function addToCart(item) {
  cart.push(item);
  EventBus.publish('cart:changed', { items: cart, count: cart.length });
}

// Badge component — subscribes (doesn't know about Header)
const unsub = EventBus.subscribe('cart:changed', ({ count }) => {
  badge.textContent = count;
});

// Sidebar component — also subscribes (doesn't know about Header)
EventBus.subscribe('cart:changed', ({ items }) => {
  renderCartPreview(items);
});
```

Observer vs Pub/Sub:

In Observer, the cart object would directly hold references to the badge and sidebar components. In Pub/Sub, the cart just publishes to `'cart:changed'` — it has no idea who's listening. This decoupling makes Pub/Sub better for large applications where components shouldn't have direct dependencies on each other.

Where Pub/Sub appears in frontend:

- Custom event buses for cross-component communication
- `BroadcastChannel` for cross-tab messaging
- Redux (actions are published, reducers subscribe)

- WebSocket message routing (messages arrive on one channel, dispatched to multiple handlers)
- Analytics event systems

When Pub/Sub causes problems: Over-using it creates "event spaghetti" — data flows through invisible channels, making it hard to trace where a change originates. For most React applications, lifting state up or using context is more explicit and debuggable than a global event bus.

Q 9.9 What is the Mediator pattern?

Difficulty: Mid **Asked at:** Google, Microsoft

Mediator centralizes communication between components. Instead of components talking directly to each other (N×N connections), they all talk through one mediator (N connections).

JAVASCRIPT Copy

```
class ChatRoom {
  constructor() {
    this.users = new Map();
  }

  register(user) {
    this.users.set(user.name, user);
    user.chatRoom = this;
  }

  send(message, from, to) {
    if (to) {
      // Direct message
      this.users.get(to)?.receive(message, from);
    } else {
      // Broadcast to all except sender
      this.users.forEach((user, name) => {
        if (name !== from) user.receive(message, from);
      });
    }
```

```
class User {
  constructor(name) {
    this.name = name;
    this.chatRoom = null;
  }

  send(message, to) {
    this.chatRoom.send(message, this.name, to);
  }

  receive(message, from) {
    console.log(`${this.name} received from ${from}: ${message}`);
  }
}

const room = new ChatRoom();
const alice = new User('Alice');
const bob = new User('Bob');

room.register(alice);
room.register(bob);

alice.send('Hey Bob!', 'Bob');     // Direct message
alice.send('Hello everyone!');      // Broadcast
```

Alice doesn't have a reference to Bob. She sends through the room (mediator), which handles routing. Adding a new user doesn't require changing any existing user's code.

Frontend examples: Form coordinators (when field A changes, fields B and C update), dialog managers (opening dialog X closes dialog Y), and state management libraries (the store mediates between actions and UI updates).

Q 9.10 What is the Strategy pattern? Give a frontend example.

Difficulty: Mid **Asked at:** Amazon, Flipkart, Razorpay, Swiggy

Strategy lets you swap algorithms or behaviors at runtime without changing the code that uses them. Instead of if/else chains, you define each strategy as a separate function and pick the right one dynamically.

JAVASCRIPT Copy

```javascript
// Without Strategy — growing if/else chain
function validateField(value, type) {
  if (type === 'email') {
    return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value);
  } else if (type === 'phone') {
    return /^\+?[\d\s-]{10,}$/.test(value);
  } else if (type === 'password') {
    return value.length >= 8 && /[A-Z]/.test(value) && /\d/.test(value);
  } else if (type === 'url') {
    // ... more branches
  }
}

// With Strategy — each validator is independent
const validators = {
  email: (value) => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value),
  phone: (value) => /^\+?[\d\s-]{10,}$/.test(value),
  password: (value) => value.length >= 8 && /[A-Z]/.test(value) && /\d/.te
  url: (value) => { try { new URL(value); return true; } catch { return fa
};

function validateField(value, type) {
  const validate = validators[type];
  if (!validate) throw new Error(`Unknown validator: ${type}`);
  return validate(value);
}

// Adding a new type doesn't touch existing code
validators.creditCard = (value) => /^\d{16}$/.test(value.replace(/\s/g, ''
```

Sorting strategies:

JAVASCRIPT Copy

```
  'price-desc': (a, b) => b.price - a.price,
  'name': (a, b) => a.name.localeCompare(b.name),
  'rating': (a, b) => b.rating - a.rating,
  'newest': (a, b) => new Date(b.createdAt) - new Date(a.createdAt),
};

function sortProducts(products, strategy) {
  return [...products].sort(sortStrategies[strategy]);
}

// In React
function ProductList({ products, sortBy }) {
  const sorted = useMemo(
    () => sortProducts(products, sortBy),
    [products, sortBy]
  );
  return sorted.map(p => <ProductCard key={p.id} product={p} />);
}
```

When to recognize Strategy: Any time you have a switch/case or if/else that selects between different algorithms — sorting, validation, formatting, pricing rules, authentication methods, rendering modes.

Q 9.11 What is the Command pattern? How do you implement undo/redo?

Difficulty: Senior **Asked at:** Google, Amazon, Microsoft, Atlassian

Command encapsulates an action as an object with `execute` and `undo` methods. This makes actions reversible, queueable, and loggable.

JAVASCRIPT Copy

```
class CommandManager {
  constructor() {
    this.history = [];
```

```
  execute(command) {
    command.execute();
    this.history.push(command);
    this.undoneCommands = [];  // Clear redo stack on new action
  }

  undo() {
    const command = this.history.pop();
    if (command) {
      command.undo();
      this.undoneCommands.push(command);
    }
  }

  redo() {
    const command = this.undoneCommands.pop();
    if (command) {
      command.execute();
      this.history.push(command);
    }
  }
}

// Text editor commands
class InsertTextCommand {
  constructor(editor, text, position) {
    this.editor = editor;
    this.text = text;
    this.position = position;
  }

  execute() {
    this.editor.insertAt(this.position, this.text);
  }

  undo() {
    this.editor.deleteAt(this.position, this.text.length);
  }
}

class DeleteTextCommand {
```

```
    this.length = length;
    this.deletedText = null;
  }

  execute() {
    this.deletedText = this.editor.getTextAt(this.position, this.length);
    this.editor.deleteAt(this.position, this.length);
  }

  undo() {
    this.editor.insertAt(this.position, this.deletedText);
  }
}

// Usage
const manager = new CommandManager();
const editor = new TextEditor();

manager.execute(new InsertTextCommand(editor, 'Hello', 0));
manager.execute(new InsertTextCommand(editor, ' World', 5));
// Editor content: "Hello World"

manager.undo();
// Editor content: "Hello"

manager.redo();
// Editor content: "Hello World"
```

Where Command pattern appears in frontend:

- Text/image editors (undo/redo)
- Drawing applications (Figma, Excalidraw)
- Spreadsheets (undoable cell edits)
- Form wizards (go back to previous step with preserved state)
- Keyboard shortcut systems (Ctrl+Z maps to undo command)

Follow-up: "How would you limit the undo history to prevent memory issues?"

only the diff or compressed version instead of the full state.

Q 9.12 What is the Iterator pattern?

Difficulty: Mid **Asked at:** Google, Amazon

Iterator provides a way to access elements of a collection sequentially without exposing its internal structure. JavaScript has this built into the language with `Symbol.iterator` and the `for ... of` loop.

JAVASCRIPT Copy

```
// Custom iterable — paginated API results
class PaginatedResults {
  constructor(fetchPage) {
    this.fetchPage = fetchPage;
  }

  async *[Symbol.asyncIterator]() {
    let page = 1;
    let hasMore = true;

    while (hasMore) {
      const response = await this.fetchPage(page);
      yield* response.items;   // Yield each item individually
      hasMore = response.hasNextPage;
      page++;
    }
  }
}

// Usage — iterate through all pages transparently
const products = new PaginatedResults(
  (page) => fetch(`/api/products?page=${page}`).then(r => r.json())
);

for await (const product of products) {
  renderProduct(product);
```

Generators (`function*`) and async generators (`async function*`) are the modern way to implement iterators in JavaScript. They're lazy — values are produced on demand, not all at once.

Q 9.13 What is the State pattern? How does it relate to state machines?

Difficulty: Senior **Asked at:** Google, Razorpay, Amazon

State pattern lets an object change its behavior when its internal state changes. Instead of large switch statements checking `this.state` , each state is a separate object with its own behavior.

JAVASCRIPT Copy

```
// Without State pattern — growing switch statement
class Order {
  handleAction(action) {
    switch (this.status) {
      case 'draft':
        if (action === 'submit') this.status = 'pending';
        if (action === 'delete') this.status = 'deleted';
        break;
      case 'pending':
        if (action === 'approve') this.status = 'approved';
        if (action === 'reject') this.status = 'draft';
        break;
      case 'approved':
        if (action === 'ship') this.status = 'shipped';
        if (action === 'cancel') this.status = 'cancelled';
        break;
      // ... more cases as states grow
    }
  }
}
```

```
// Each state handles its own transitions
const orderStates = {
  draft: {
    submit: () => 'pending',
    delete: () => 'deleted',
  },
  pending: {
    approve: () => 'approved',
    reject: () => 'draft',
  },
  approved: {
    ship: () => 'shipped',
    cancel: () => 'cancelled',
  },
  shipped: {
    deliver: () => 'delivered',
  },
  delivered: {},   // Terminal state — no transitions
  cancelled: {},
  deleted: {},
};

function transition(currentState, action) {
  const handler = orderStates[currentState]?.[action];
  if (!handler) throw new Error(`Invalid action '${action}' in state '${cu
  return handler();
}
```

This is a finite state machine — a defined set of states with defined transitions. Invalid transitions are impossible (you can't ship a draft order). The state machine makes impossible states impossible.

XState — production-grade state machines for React:

JAVASCRIPT Copy

```
import { createMachine } from 'xstate';

const orderMachine = createMachine({
  id: 'order',
  initial: 'draft',
```

```
    approved: { on: { SHIP: 'shipped', CANCEL: 'cancelled' } },
    shipped: { on: { DELIVER: 'delivered' } },
    delivered: { type: 'final' },
    cancelled: { type: 'final' },
    deleted: { type: 'final' },
  },
});
```

Where State pattern shines in frontend: Multi-step forms, checkout flows, media players (playing/paused/stopped/buffering), authentication flows (logged-out/logging-in/authenticated/expired), and any UI with complex conditional logic that depends on the current state.

Q 9.14 What is the Template Method pattern?

Difficulty: Mid **Asked at:** Amazon, Microsoft

Template Method defines the skeleton of an algorithm, letting subclasses override specific steps without changing the structure.

JAVASCRIPT Copy

```
// Base data fetcher with a defined workflow
class DataFetcher {
  async fetch(url) {
    const rawData = await this.makeRequest(url);      // Step 1: Fetch
    const parsed = this.parseResponse(rawData);       // Step 2: Parse
    const validated = this.validate(parsed);          // Step 3: Validate
    return this.transform(validated);                 // Step 4: Transfor
  }

  async makeRequest(url) { return fetch(url).then(r => r.json()); }
  parseResponse(data) { return data; }  // Override in subclass
  validate(data) { return data; }       // Override in subclass
  transform(data) { return data; }      // Override in subclass
}
```

```
class JsonApiFetcher extends DataFetcher {
  parseResponse(data) { return data.results; }
  validate(data) { return data.filter(item => item.id && item.name); }
  transform(data) { return data.map(item => ({ ...item, source: 'api' })); }
}

// CSV fetcher
class CsvFetcher extends DataFetcher {
  async makeRequest(url) { return fetch(url).then(r => r.text()); }
  parseResponse(text) { return text.split('
').map(row => row.split(',')); }
  validate(rows) { return rows.filter(row => row.length === 3); }
}
```

The algorithm structure (`fetch → parse → validate → transform`) is fixed. Each fetcher customizes individual steps. In modern JavaScript, you'd often use composition (passing functions) instead of inheritance, but the concept — a fixed workflow with customizable steps — remains relevant.

Q 9.15 What is the Chain of Responsibility pattern?

Difficulty: Senior **Asked at:** Amazon, Google, Microsoft

Chain of Responsibility passes a request through a chain of handlers. Each handler decides whether to process the request or pass it to the next handler. Express.js middleware is the classic frontend-adjacent example.

JAVASCRIPT Copy

```
// Middleware pipeline (Express-style)
class Pipeline {
  constructor() {
    this.middlewares = [];
  }

  use(fn) {
```

```
    this.middlewares.push(fn);
    return this;
  }

  async execute(context) {
    let index = 0;

    const next = async () => {
      if (index < this.middlewares.length) {
        const middleware = this.middlewares[index++];
        await middleware(context, next);
      }
    };

    await next();
    return context;
  }
}

// Usage — request processing pipeline
const api = new Pipeline();

api.use(async (ctx, next) => {
  ctx.startTime = Date.now();
  await next();
  console.log(`Request took ${Date.now() - ctx.startTime}ms`);
});

api.use(async (ctx, next) => {
  if (!ctx.headers.authorization) {
    ctx.status = 401;
    return;  // Don't call next — stop the chain
  }
  ctx.user = await verifyToken(ctx.headers.authorization);
  await next();
});

api.use(async (ctx, next) => {
  ctx.body = await fetchData(ctx.user.id);
  await next();
});

api.execute({ headers: { authorization: 'Bearer token123' } });
```

happen before data fetching.

Frontend examples: Form validation chains (check required → check format → check server-side), error handling pipelines, and request/response interceptors in API clients like Axios.

QUICK CHECK — no marks, just for you.

1. According to the lesson, what structurally distinguishes Pub/Sub from the Observer pattern?

- ○ Pub/Sub requires a network connection such as WebSocket, while Observer works only within a single JavaScript module.
- ○ Pub/Sub puts a message broker between publishers and subscribers, so neither side holds a reference to the other, while in Observer the subject knows its observers directly.
- ○ Pub/Sub delivers events asynchronously while Observer always delivers them synchronously.
- ○ Pub/Sub allows many observers per subject while Observer supports only a single observer at a time.

Check answer

2. Using the Mediator (ChatRoom) implementation from the lesson, what does this snippet log?

JAVASCRIPT Copy

```
class ChatRoom {
  constructor() { this.users = new Map(); }
  register(user) { this.users.set(user.name, user); user.chatRoom = this; }
  send(message, from, to) {
    if (to) {
```

```
        if (name !== from) user.receive(message, from);
      });
    }
  }
}

class User {
  constructor(name) { this.name = name; this.chatRoom = null; }
  send(message, to) { this.chatRoom.send(message, this.name, to); }
  receive(message, from) { console.log(`${this.name} received from ${from}:
}

const room = new ChatRoom();
const alice = new User('Alice');
const bob = new User('Bob');
const carol = new User('Carol');

room.register(alice);
room.register(bob);
room.register(carol);

alice.send('Hey Bob!', 'Bob');
alice.send('Hello everyone!');
```

- Bob received from Alice: Hey Bob!
  Alice received from Alice: Hello everyone!
  Bob received from Alice: Hello everyone!
  Carol received from Alice: Hello everyone!
- TypeError
- Bob received from Alice: Hey Bob!
  Carol received from Alice: Hey Bob!
  Bob received from Alice: Hello everyone!
  Carol received from Alice: Hello everyone!
- Bob received from Alice: Hey Bob!
  Bob received from Alice: Hello everyone!
  Carol received from Alice: Hello everyone!

3. In the lesson's State-pattern version of the order flow, the transition function looks up orderStates[currentState]?.[action]. What happens when you call transition('draft', 'ship'), and why is that the point of the pattern?

- ○ It throws an error, because 'draft' defines no 'ship' transition — the state machine makes invalid transitions impossible rather than silently allowing them.
- ○ It returns 'draft' unchanged, because the pattern treats unknown actions as no-ops so the UI never crashes.
- ○ It returns 'shipped', because the transition map falls back to the closest matching state's handlers.
- ○ It returns undefined and leaves the order in 'draft', because optional chaining swallows the missing handler.

Check answer

‹ Previous
Creational patterns

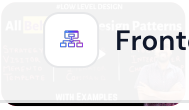
Next ›
Structural patterns

LEARN MORE

ARTICLES & DOCS

Behavioral patterns (Refactoring.Guru)
refactoring.guru

VIDEOS



Frontend System Desi...

YouTube



Behavioral patterns (LLD)

YouTube



The observer pattern explained

YouTube

Hand-picked references for this lesson.